GCP Deployment Scenarios and Architecture Decisions (One-Page)

Objective

Deliver the same online boutique application across three deployment scenarios on GCP while keeping checkout reliable, data durable, and operations practical for different scale and team maturity levels.

Terminology (Quick Clarifications)

- Sidecar: an additional container running in the same runtime boundary as a primary service container. It provides supporting capabilities (for example, database connectivity proxy or co-located dependency) while communicating over localhost.
- RBAC: Role-Based Access Control.
- TLS: Transport Layer Security.
- IAM: Identity and Access Management.
- Pub/Sub: Publish/Subscribe messaging.

Application Components

- User-facing: frontend
- Core commerce services: productcatalogservice, cartservice (with redis-cart), recommendationservice, checkoutservice, paymentservice, shippingservice, currencyservice, emailservice, adservice
- Data services: Cloud SQL for PostgreSQL (transactional orders), Firestore (inventory metadata/state)
- Eventing: Google Cloud Pub/Sub topic order-notifications
- Additional serverless components: Cloud Run service order-workflow, Cloud Function subscriber for notifications/workflows

Scenario 1: VM Deployment (Compute Engine + Docker Compose)

- Runtime model: all microservices run as containers on one VM.
- Networking: internal service-to-service calls on compose network names and ports.
- Data path: checkoutservice writes to Cloud SQL (private IP or Cloud SQL proxy path).
- Best fit: low complexity, lower traffic, fast debugging, direct host control.
- Tradeoff: manual scaling, patching, and host lifecycle ownership.

Scenario 2: Container Platform Deployment (GKE)

- Runtime model: microservices as Kubernetes Deployments/Services.

- Networking: Kubernetes service discovery and cluster networking.
- Data path: checkoutservice uses Cloud SQL Auth Proxy sidecar for Cloud SQL access.
- Best fit: production-grade orchestration, rolling updates, fine-grained controls.
- Tradeoff: highest ops overhead (cluster lifecycle, Role-Based Access Control (RBAC), node and policy management).

Scenario 3: Serverless Platform Deployment (Cloud Run)

- Runtime model: either single service (for order-workflow) or multi-container sidecar service for boutique components.
- Networking:
  - Sidecar model: localhost ports between containers, Transport Layer Security (TLS) disabled for in-process hop.
  - Split-service model: service.run.app:443 endpoints, Transport Layer Security (TLS) enabled.
- Data path: order-workflow on Cloud Run writes to Cloud SQL, updates Firestore, and publishes Google Cloud Pub/Sub events.
- Best fit: variable traffic, rapid delivery, minimal infrastructure management.
- Tradeoff: per-revision container limits, stricter runtime constraints, careful env/port/Transport Layer Security (TLS) alignment required.

Why These Architecture Decisions Were Made

1. Cloud SQL (PostgreSQL) for order records

- Reason: checkout and order history are strongly relational and require transactional integrity (order header + order items), SQL queryability, and predictable consistency.
- Result: normalized tables (customer_order, order_item) and ACID semantics for financial events.

2. Firestore for inventory metadata

- Reason: inventory and product state updates are document-shaped and evolve quickly; schema flexibility and low-latency key access are valuable.
- Result: simple product-centric reads/writes (inventory/{product_id}) without relational migration friction.

3. Why AlloyDB, MySQL, or SQL Server were not selected

- AlloyDB (PostgreSQL-compatible): strong option, but this project did not require its premium performance profile or operational features at current scale; Cloud SQL for PostgreSQL provided sufficient transactional capability with lower complexity and cost for the assignment scope.

- Cloud SQL for MySQL: technically feasible, but the existing schema and scripts align with PostgreSQL behavior and ecosystem; switching engines would add migration and compatibility effort without clear functional benefit.
- Cloud SQL for SQL Server: not chosen due to higher licensing/cost considerations and unnecessary platform coupling for this microservices demo workload.
- Result: Cloud SQL for PostgreSQL offered the best fit for compatibility, predictable operations, and rubric-focused implementation speed.

4. Pub/Sub for decoupling post-checkout actions

- Reason: order completion should not block on downstream notification or enrichment steps.
- Result: asynchronous, resilient event-driven fanout with retry semantics.

5. Additional serverless service (Cloud Function subscriber)

- Reason: notification, audit, and downstream integration logic is event-driven and bursty; serverless is cost-efficient and operationally light.
- Result: Cloud Function scales on demand from Pub/Sub events and keeps checkout latency stable.

Pub/Sub Invocation Point and Embedding

- Is Pub/Sub invoked during checkout: yes, but after order placement has been accepted and persisted by order-workflow.
- Which service publishes: order-workflow (Cloud Run) publishes ORDER_CREATED to the order-notifications topic.
- Is Pub/Sub embedded in checkoutservice: no. checkoutservice calls order-workflow; order-workflow owns publish logic so checkout can stay focused on transaction orchestration.
- Why this matters: checkout latency remains stable because notification delivery is asynchronous.

What "Order Notification" Means Here

- An order notification is an event payload (for example ORDER_CREATED with order_id, user_email, totals, and items) emitted to Pub/Sub.
- Consumers (such as a Cloud Function) process that event for non-blocking downstream actions: confirmation email, audit entry, analytics stream update, fraud rules, or fulfillment integration.
- It is a technical event contract, not only an end-user email.

Serverless Additional Service Interaction Across Scenarios

Common pattern used by VM, GKE, and Cloud Run storefront variants: 1. User checks out via frontend and checkoutservice. 2. checkoutservice invokes

order-workflow /order-events. 3. order-workflow persists order to Cloud SQL and updates Firestore. 4. order-workflow publishes ORDER_CREATED to Pub/Sub. 5. Cloud Function subscriber consumes event and performs email/SMS/CRM/audit actions.

Because this integration is event-driven and endpoint-based, the same serverless function can support all three deployment scenarios without changing core checkout behavior.

Why Markdown (.md) Is Good for This Deliverable

- Portable and lightweight plain text (easy to version in Git and review in pull requests).
- Human-readable in raw form and well-rendered in GitHub/GitLab/VS Code.
- Fast to update during architecture changes without proprietary tool lock-in.
- Easy to export to Word/PDF when submission format requires it.

Rubric Coverage (Demonstrated)

- Architecture clarity: three deployment topologies and component boundaries are explicit.
- Data design rationale: relational vs document storage justified by workload shape.
- Security model: private DB connectivity, secret management, Identity and Access Management (IAM)-based service access.
- Reliability: async eventing, retries, and reduced checkout critical path dependencies.
- Scalability: VM (vertical/manual), GKE (horizontal/orchestrated), Cloud Run/Functions (autoscale).
- Cost/operations: VM lowest abstraction, GKE highest control/overhead, serverless best for elastic demand.
- Integration quality: one event contract supports all scenarios consistently.
- Maintainability: shared business flow with environment-specific runtime choices.